

torch.fft.rfftfreq#

<https://pytorch.org/docs/stable/generated/torch.fft.rfftfreq.html>

Description:

Computes the sample frequencies for `rfft()` with a signal of size `n`. `rfft()` returns Hermitian one-sided output, so only the positive frequency terms are returned. For a real FFT of length `n` and with inputs spaced in length unit `d`, the frequencies are: For even lengths, the Nyquist frequency at $f[n/2]$ can be thought of as either negative or positive. Unlike `fftfreq()`, `rfftfreq()` always returns it as positive.

Function Signature

```
torch.fft.rfftfreq(n, d=1.0, *, out=None, dtype=None,
layout=torch.strided, device=None, requires_grad=False) →
Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

out (Tensor, optional) – the output tensor. dtype (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, uses a global default (see torch.set_default_dtype()). layout (torch.layout, optional) – the desired layout of returned Tensor. Default: torch.strided. device (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see torch.set_default_device()). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types. requires_grad (bool, optional) – If autograd should record operations on the returned tensor. Default: False.

Code Examples

```
f = torch.arange((n + 1) // 2) / (d * n)
```

```
>>> torch.fft.rfftfreq(5)
tensor([0.0000, 0.2000, 0.4000])
```

```
>>> torch.fft.rfftfreq(4)
tensor([0.0000, 0.2500, 0.5000])
```

Notes & Warnings

NOTE

Note `rfft()` returns Hermitian one-sided output, so only the positive frequency terms are returned. For a real FFT of length n and with inputs spaced in length unit d , the frequencies are: $f = \text{torch.arange}((n + 1) // 2) / (d * n)$

NOTE

Note For even lengths, the Nyquist frequency at $f[n/2]$ can be thought of as either negative or positive. Unlike `fftfreq()`, `rfftfreq()` always returns it as positive.

Detailed Documentation

Documentation

Computes the sample frequencies for `rfft()` with a signal of size n .

`rfft()` returns Hermitian one-sided output, so only the positive frequency terms are returned. For a real FFT of length n and with inputs spaced in length unit d , the frequencies are:

For even lengths, the Nyquist frequency at $f[n/2]$ can be thought of as either negative or positive. Unlike `fftfreq()`, `rfftfreq()` always returns it as positive.

n (int) – the real FFT length

`d` (float, optional) – The sampling length scale. The spacing between individual samples of the FFT input. The default assumes unit spacing, dividing that result by the actual spacing gives the result in physical frequency units.

`out` (Tensor, optional) – the output tensor.

`dtype` (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, uses a global default (see `torch.set_default_dtype()`).

`layout` (torch.layout, optional) – the desired layout of returned Tensor. Default: `torch.strided`.

`device` (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see `torch.set_default_device()`). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types.

`requires_grad` (bool, optional) – If autograd should record operations on the returned tensor. Default: False.

Compared to the output from `fftfreq()`, we see that the Nyquist frequency at `f[2]` has changed sign: `>>> torch.fft.fftfreq(4) tensor([ 0.0000, 0.2500, -0.5000, -0.2500])`